Announcements

- Portfolio Assignment 2 is due **Saturday 11:59PM**

Web Engineering & Development (SWE 363)

Interactive Front-End Development

React Sample Projects

Github Repository

In today's lecture:

- Forms
- Router

Reference:

- Zybook: 5.12-5.14

5.12 Forms (React)

What are React Forms?

React Forms are controlled components where React manages the input values through state.

Key Concepts:

- **Controlled Components** - React controls input values
- **State Management** - `useState` tracks form data
- **Validation** - Check data before submission
- **Event Handling** - Process form submission

Controlled vs Uncontrolled Components

Uncontrolled (Traditional HTML)

```
<input type="text" />  // Browser manages value
```

Controlled (React Way)

```
const [email, setEmail] = useState("");

<input
  type="email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

Why Use Controlled Components?

Advantages:

- **Single source of truth** - State controls the UI
- **Easy validation** - Check state before submission
- **Predictable behavior** - Same across all browsers
- **Dynamic updates** - UI updates as user types

Without controlled components:

- Hard to validate
- Inconsistent behavior
- Difficult to reset forms

Basic Form Structure

```
import { useState } from "react";

function Registration() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [gender, setGender] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page reload
    // Handle form submission
  };
  return (
    <form onSubmit={handleSubmit}>
      {/* Form fields go here */}
      <button type="submit">Register</button>
    </form>
  );
}
```

Form Validation

Why Validate?

- **User Experience** - Show helpful error messages
- **Data Quality** - Ensure correct information
- **Security** - Prevent invalid submissions

Common Validation Rules:

- **Required fields** - Must not be empty
- **Email format** - Contains "@" and ends with ".com"
- **Password strength** - Minimum length, special characters
- **Radio selection** - Must choose an option

Form - Validation Implementation

```
const [errors, setErrors] = useState({});
const handleSubmit = (e) => {
  e.preventDefault();

  const nextErrors = {};
  // Email validation
  // ...

  // Password validation
  if (!password.trim()) {
    nextErrors.password = "Password is required";
  }
  setErrors(nextErrors);
  if (Object.keys(nextErrors).length > 0) return;

  alert(`User Registered: ${email}`); // Success! Show alert
};
```

Form - Displaying Error Messages

Error State

```
const [errors, setErrors] = useState({});
```

Error Display

```
{errors.email && <p className="error">{errors.email}</p>}
{errors.password && <p className="error">{errors.password}</p>}
{errors.gender && <p className="error">{errors.gender}</p>}
```

Form - Displaying Error Messages

Complete Form Field with Error

```
<div className="form-row">
  <label htmlFor="email">Email</label>
  <input
    id="email"
    type="email"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />
  {errors.email && <p className="error">{errors.email}</p>}
</div>
```

Form Submission Flow

Step-by-Step Process:

1. **User fills form** → State updates
2. **User clicks submit** → `handleSubmit` runs
3. **Prevent default** → `e.preventDefault()`
4. **Validate data** → Check all fields
5. **Show errors** → Display validation messages
6. **Success action** → Show alert, reset form, navigate

Disabling Form Submit Button

Conditional Button State

```
<button
  type="submit"
  disabled={!email || !password || !gender}
>
  Register
</button>
```

Why Disable?

- **Prevents empty submissions**
- **Better UX** - Shows when form is ready
- **Visual feedback** - Button appears inactive

5.14 Router

What is React Router?

React Router is a library that enables **client-side routing** in React applications.

Key Benefits:

- **No page reloads** - Faster navigation
- **URL as state** - URLs represent user location
- **Better UX** - Smooth transitions between pages

Think of it as:

> A **traffic controller** that decides which component to show based on the URL

Traditional Web Navigation vs React Router

Traditional Web (Server-Side)

```
User clicks link → Browser requests new page → Server sends HTML → Page reloads
```

React Router (Client-Side)

```
User clicks link → URL changes → React shows new component → No reload!
```

Basic Router Setup

Step 1: Install React Router

```
npm i react-router-dom
```

Step 2: Import Router Components

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
```

Step 3: Wrap Your App

```
<BrowserRouter>
  <App />
</BrowserRouter>
```

Creating Routes

Basic Route Structure

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/registration" element={<Registration />} />
  <Route path="*" element={<h2>404 – Not Found</h2>} />
</Routes>
```

Key Points:

- `path="/"` - Home page (exact match)
- `path="*"` - Catch-all for 404 pages
- `element={<Component />}` - What to render

Navigation Using Link Component

Basic Link Syntax

```
<Link to="/">Home</Link>
<Link to="/about">About</Link>
<Link to="/registration">Registration</Link>
```

Key Features:

- **Changes URL** without page reload
- **Triggers route rendering** - Shows matching component
- **Simple navigation** - Basic link functionality

30m

Demo: Student Registration Portal

5.5 React Advance

What we'll build:

- **3 pages** with React Router
- **Registration form** with validation
- **Error handling** and success messages

Common Mistakes to Avoid

Router Mistakes:

- Forgetting to wrap app in `BrowserRouter`
- Using `href` instead of `to` prop
- Not importing required components

Form Mistakes:

- Not using `e.preventDefault()`
- Forgetting to validate before submission
- Not handling empty state properly

Next Class

- More on React